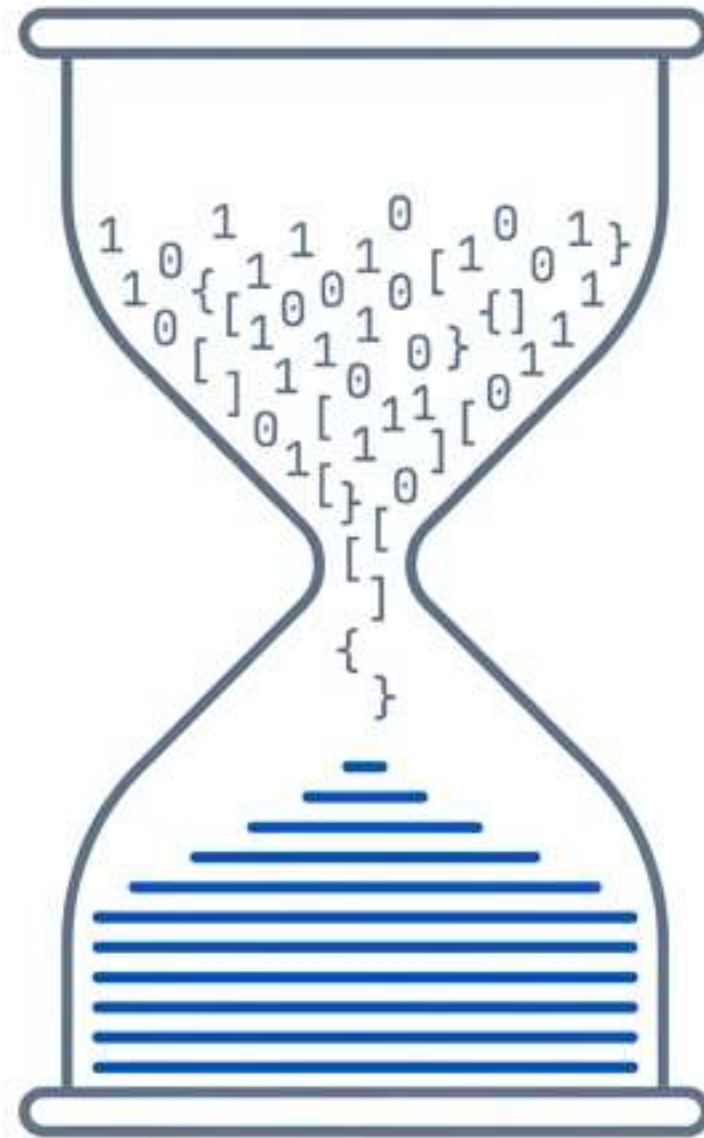


Mastering Synchronisation in Selenium

A Deep Dive into Explicit Waits



Overview: The Need for Speed Control

The Challenge: Async Mismatch

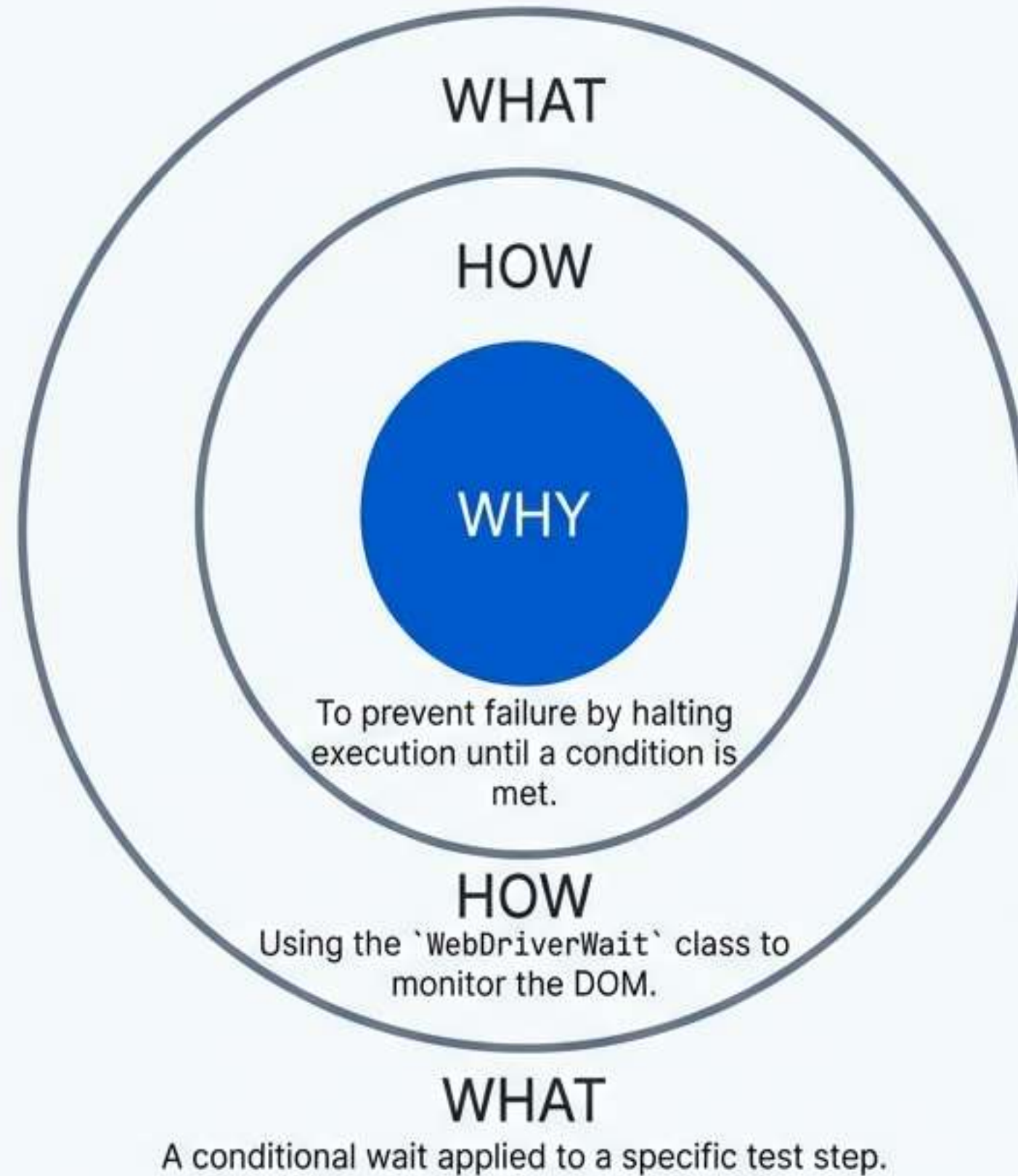


The Solution: Synchronisation



- ❖ **The Problem:** Automation scripts run in milliseconds; web apps load in seconds. This gap causes `NoSuchElementException` exceptions.
- ❖ **The Fix:** Explicit Waits align the WebDriver's speed with the browser's state.
- ❖ **Focus:** We will master **Explicit Waits**—halting execution until specific conditions are met.

Core Concepts: The Golden Circle



Syntax

Driver (Control)

Passes browser control to the wait object.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));  
wait.until(ExpectedConditions.visibilityOf(element));
```

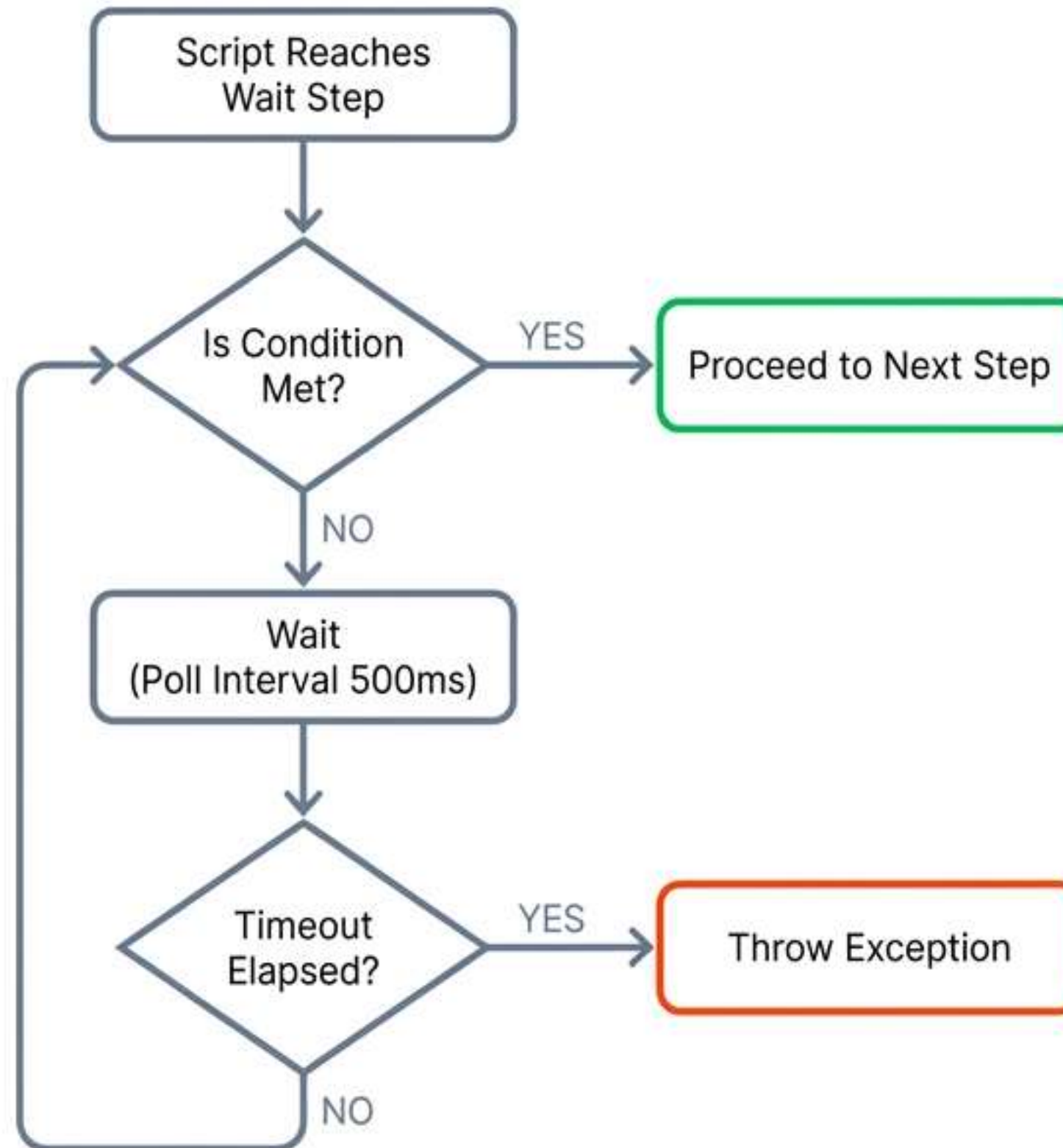
Timeout (Max Wait)

Max time to wait before throwing an exception.

The Trigger

The specific state required to proceed.

Process Workflow: The Polling Mechanism



****Dynamic Waiting:**** Unlike `Thread.sleep()`, Explicit Wait exits the loop immediately when the condition is true. It does not wait the full 10 seconds if the element is ready.

****Dynamic Waiting:**** Unlike `Thread.sleep()`, Explicit Wait exits the loop immediately when the condition is true. It does not wait the full 10 seconds if the element is ready in 2.

Real-World Practical Scenarios



Interactions

`elementToBeClickable`

Context

Ensures a button is active (not disabled) before clicking. Critical for forms.

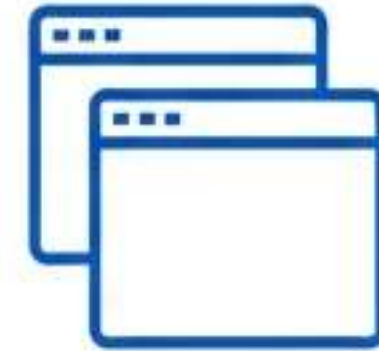


Visual Validation

`visibilityOf /`
`invisibilityOf`

Context

Waiting for loading spinners to disappear or success messages to appear.



Window Management

`numberOfWindowsToBe`

Context

Validating a new tab has opened before switching driver context.

Best Practices

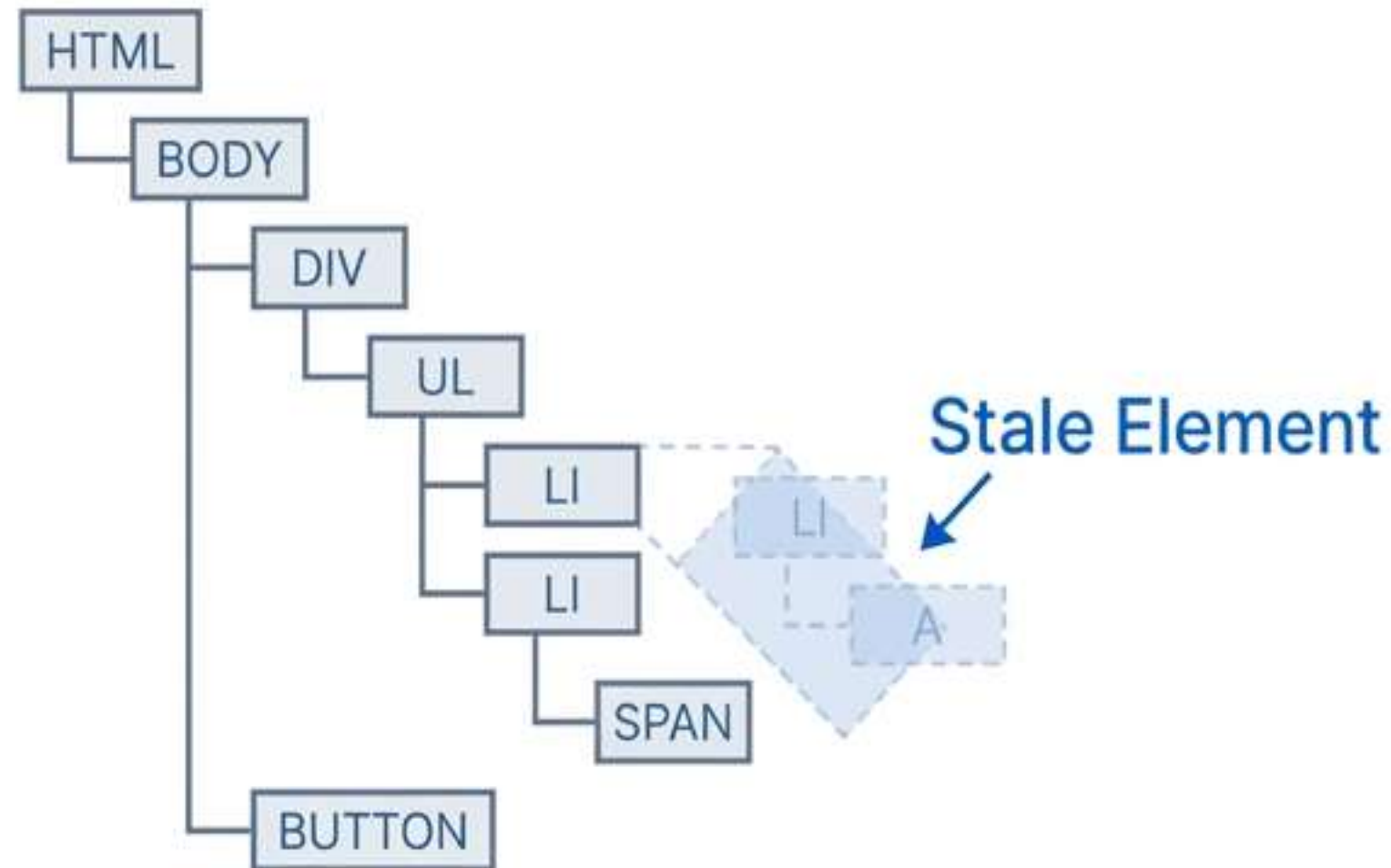
✓ DO

- Use specific conditions (e.g., `invisibilityOf`) for precise control.
- Define reasonable timeouts (10-30s) based on app performance.
- Handle exceptions (`TimeoutException`) gracefully.

✗ DON'T

- Avoid mixing Implicit and Explicit Waits. This causes unpredictable 'zombie' wait times.
- Never use `Thread.sleep()`. It is static, slow, and ignores the browser state.

Advanced Challenges: The DOM



StaleElementReferenceException

Issue: The element is found, but the DOM updates before the action occurs.

Solution: Implement 'Refreshed Waiting' to re-find the element within the loop.

False Positives

Issue: An element is present in the HTML but hidden (0x0 size) from the user.

Solution: Always prefer 'visibilityOf' over 'presenceOf'.

Interview Preparation

Q: Implicit vs. Explicit Wait?

A: Implicit is global (all elements). Explicit is local (specific elements/conditions). **Never mix them.**

Q: Handling Hidden Loaders?

A: Use `invisibilityOf` for the loader, then `elementToBeClickable` for the target.

Bonus Topic: **Fluent Wait** allows custom polling intervals and ignoring specific exceptions.

Key Takeaways



Precision

Target specific elements and states.



Efficiency

Exits wait immediately upon success.



Robustness

Prevents flaky tests caused by lag.



Flexibility

Vast library of conditions (Alerts, Frames, Titles).